



Computer Games Development CW208
Project Report
Year IV

Emily Breen
C00288668
11/04/2026

Acknowledgements

I would like to sincerely thank the following people for their support and guidance throughout this project.

Firstly, to my supervisor, Amr Abdelhafez, for his consistent support and guidance throughout the entire development process. Our weekly meetings were invaluable, and his encouragement at each stage helped shape both the project and my approach to it.

I would also like to thank Omer Ali for his positive attitude and genuine enthusiasm towards my project. His feedback during the presentation was incredibly valuable. I am also grateful to Noel O'Hara and Martin Harrington, whose feedback encouraged me to reflect on aspects of the game I had not previously considered, ultimately helping to improve it.

To my classmates, Stephen Dunne and Ryan Holloway, thank you for your constant support and understanding throughout this project. Your encouragement, patience, and willingness to take the time to test my game and give honest feedback truly meant a lot to me, especially during the more difficult moments.

Finally, to my partner, Bridget Dempsey, thank you for being my rock throughout my entire college journey. Your constant encouragement, patience, and support made this possible, and I truly would not have come this far without you.

Project Abstract

This project presents *Can I Hinder?* a crowd-interactive 2.5 action adventure game that explores the integration of real time spectator input into a traditionally single-player experience. The system is built across multiple components, including a C++ SFML game client, a WebSocket relay server, A REST API and a Progressive Web App (PWA), all working together to create a live, reactive gameplay environment.

The core Idea behind this project is to introduce unpredictability and emergent behaviour through audience participation. Spectators are able to influence game play in real-time by choosing to either assist the player through buffs or hinder their progress through debuffs and environmental challenges. This transforms the experience from a static single-player journey into a dynamic system shaped by both the player and the audience.

This project demonstrates how real-time networking and modern web technologies can be combined with game design to create engaging and interactive experiences. It also highlights the technical and design challenges involved in synchronising multiple systems, maintaining performance and balancing player control with external influences.

Project Introduction and Research Question

The central question behind this project is whether real-time audience interaction can meaningfully enhance a single player experience. Traditional action-adventure games typically follow a structured and predictable design, where the player progresses through challenges with limited external influence. While this allows for controlled gameplay, it often lacks the unpredictability and variation seen in live, shared experiences.

Can I Hinder? was developed to explore how this model can be extended by introducing a live spectator interaction layer. By allowing external participants to connect to the game through a web-based interface and influence gameplay in real-time, the project aims to transform a single-player experience into a dynamic shared system.

This project was motivated by the growing popularity of live-streamed gameplay and interactive media, where audiences are no longer passive viewers but active contributors. Platforms that support real-time interaction have demonstrated that audience participation can significantly shape player experience, but integrating this into a structured game environment presents both technical and design challenges.

The primary research question explored in this project is:

Can real-time spectator interaction, enhance player engagement and introduce meaningful variation in a single player action-adventure game?

To address this question, the project aims to design and implement a fully playable game supported by a multi-component architecture, while also exploring how such a system can be balanced to ensure that spectator influence enhances, rather than disrupts, the core gameplay experience.

Literature Review

The WebSocket protocol (RFC 6455, 2011) was the main thing I used for real-time communication in this project. WebSockets allow continuous two-way communication between clients, which makes them ideal for systems where low latency is important. This is used directly in *Can I Hinder?*, where spectator actions need to be sent to the game and applied instantly.

Coulouris et al. (2011) helped shape how I structured the system. Their work on distributed systems explains how a system can be split into separate parts that communicate over a network. This is reflected in the project, which is made up of a C++ game client, a Node.js relay server, a REST API, and a PWA. Splitting it this way made it easier to manage each part while still keeping everything connected in real time.

Gregory's *Game Engine Architecture* (2014) influenced how I designed the game client. In particular, the idea of keeping systems modular helped when organising things like input handling, rendering, and AI. This made it much easier to add new features without breaking existing systems.

Enemy movement is based on steering behaviours introduced by Reynolds (1999) and later expanded by Millington and Funge (2009). These behaviours, such as Seek, Arrive, Pursue, Wander, and Flee, are a simple way of creating more natural enemy movement. I implemented these in a separate *AIBehaviour* class, along with wall avoidance and separation, so they could be adjusted without affecting other parts of the game.

The idea of crowd interaction comes from systems like *Twitch Plays Pokémon* (2014), where large numbers of players can control a game together. These systems are usually very chaotic, so in this project I tried to make it more structured by introducing a help vs hinder system, along with progression-based unlocks to reward spectators who stay engaged.

The React documentation (React Team, 2024) was used when building the PWA. It helped with structuring components and managing state, which was important for making sure the interface was responsive and easy to use. Using a PWA also meant spectators could join quickly without needing to install anything.

Evaluation and Discussion

The project was developed over 3 main milestones, with each phase building upon the previous one and gradually turning the system into a complete and playable experience.

The first milestone, which ran from September to Christmas, focused on building the core technical foundation of the project. This included setting up rendering, player movement, collision and the initial WebSocket communication between the game and the PWA. By the end of this phase, the game could load maps move an animated player and respond to basic spectator input, which established the core interaction that the rest of the project builds on.

The second milestone focused on expanding both the gameplay and the networking side of the system, deploying the backend and frontend services to Azure, and developing the main game play loop. The PWA was also improved to provide better user experience, making it easier for spectators to interact with the game. This phase was important in turning the project from a technical prototype into something that felt more like a complete game.

The final milestone focused on completing and refining the overall experience. This included adding visual effects, balancing the help and hinder mechanics, better UI for the PWA within the radial menu, adding a new unlockable help and hinder button after the desired amount is reached, and a turn based battle system was also introduced for the final level, changing the pace of the gameplay to a more structured encounter compared to the real-time exploration. This particular feature was implemented as it pays homage to why I wanted to pursue game development.

Major Technical Achievements

One of the main technical achievements of this project is the successful implementation of real-time communication between multiple systems. The game client, WebSocket relay server, REST API, and PWA all communicate together, allowing spectator actions to be sent and applied in the game almost instantly. Getting this working reliably without affecting gameplay performance was a key part of the project.

Another major achievement is the overall system architecture. The project is split into separate components, including the C++ SFML game, Node.js relay server, ASP.NET Core API, and React-based PWA. Designing the system in this way made it easier to develop and test each part independently, while still allowing them to work together as a single system.

The implementation of the help and hinder system is also a significant achievement. This system allows spectators to directly influence gameplay in real time, with actions that can either assist or disrupt the player. The addition of progression-based unlocks, where stronger actions become available after a certain number of inputs, helped create a more engaging and structured interaction system.

Another key technical achievement is the AI behaviour system used for enemies. Steering behaviours such as Seek, Arrive, Pursue, Wander, and Flee were implemented, along with additional features such as separation and wall avoidance. These systems allowed enemies to move in a more natural and responsive way, improving the overall gameplay experience.

The final level turn-based battle system is also an important addition. This introduced a different style of gameplay compared to the rest of the project, requiring a separate structure for handling player and enemy actions. Implementing both real-time and turn-based systems within the same project required careful design to ensure they worked together without conflict.

Finally, deploying the backend services and PWA to Azure was a key achievement. This allowed the system to move beyond a local setup and function as a fully connected, real-world application, making it possible for spectators to interact with the game remotely.

Project Review

What went wrong

One of the more significant issues encountered during development was enemy spawning, where enemies could appear outside of valid walkable areas. This was initially caused by spawning positions not being validated against the map's walkable tile data, which resulted in enemies becoming stuck or behaving incorrectly.

To address this, a dedicated spawning function was introduced within the map system. This function uses the map's walkable tile data and performs a search to identify valid spawn locations within reachable areas of the map. It also avoids spawning enemies too close to the player by applying a minimum distance check.

This solution significantly improved the reliability of enemy spawning and ensured that enemies appeared in valid and accessible positions. However, it also highlighted the importance of validating gameplay logic against environment data, particularly when working with tile-based maps and dynamic spawning systems.

Another notable issue was deploying to Azure, particularly when trying to update existing services. Initially, deployment was handled through VS Code, however updates to both the PWA and Node server were not consistently applied. This meant that even after deploying changes, the live version would not reflect the latest code. As a result, the only reliable workaround at the time was to delete and redeploy the services, which slowed down development significantly.

To resolve this, I implemented a continuous deployment pipeline using GitHub Actions. YAML workflows were created for the PWA, Node.js and the API, using Azure publish profiles stored as Github secrets. This allowed the project to automatically build and deploy whenever changes were pushed to the main branch.

This approach made deployment far more reliable and removed the need for manual redeployment.

What went right

One of the main things that went right in the project was the integration of multiple systems into a single working application. The game client, WebSocket relay server, REST API, and PWA were all able to communicate together in real time. Seeing everything connect and work together for the first time was incredibly satisfying, as this was the core goal of the project.

The help and hinder unlock system also worked very well once implemented. It created a stronger incentive for spectators to stay engaged, giving them a reason to continue interacting with the game rather than just sending occasional inputs.

As discussed above, the deployment pipeline using GitHub Actions made deployment much faster and more reliable. This allowed more time to be spent on developing the game itself rather than dealing with deployment issues.

The introduction of the turn-based battle system is one of the biggest highlights of the project. It not only adds variety to the gameplay, but also challenged me to think more carefully about balancing. Managing different abilities, magic effects, and items made this part of development particularly enjoyable and rewarding.

Conclusion

This project successfully developed a crowd-interactive system that allows spectators to influence a single player game in real-time. The final implementation demonstrates how multiple technologies can be combined to create a responsive and engaging experience, where both player and audience contribute to the outcome of the gameplay.

The integration of real-time networking, structured interaction systems and varied gameplay mechanics show that spectator input can be introduced without losing control of the core game experience.

Future Work

There are several areas where this project could be expanded further. One of the main improvements would be to introduce more varied spectator interactions, allowing a wider range of effects that could influence gameplay in different ways i.e. environmental changes or cooperative spectator abilities.

Another area for development would be the turn based battle system. While it works well as a final encounter, it could be made into a more complete system with additional mechanics, abilities, experience points and enemy types

References

Coulouris, G., Dollimore, J., Kindberg, T. and Blair, G. (2011) Distributed Systems: Concepts and Design (5th ed.). Addison-Wesley.

Gregory, J. (2018) - Game Engine Architecture (3rd ed.). CRC Press.

Millington, I. and Funge, J. (2016) - Artificial Intelligence for Games (2nd ed.). CRC Press.

Fette, I. and Melnikov, A. (2011) - The WebSocket Protocol (RFC 6455). IETF [Online]
Available at: <https://datatracker.ietf.org/doc/html/rfc6455>.

SFML Development Team. (2024). SFML 3.0 Documentation. [Online].
Available at: <https://www.sfml-dev.org/documentation/3.0/>

Boost Libraries. (2024). Boost.Beast Documentation. [Online].
Available at: <https://www.boost.org/doc/libs/release/libs/boost/>

Microsoft. (2024). ASP.NET Core 8 Documentation. [Online].
Available at: <https://learn.microsoft.com/en-us/aspnet/core/>

TMXLite. (2024). GitHub Repository. [Online].
Available at: <https://github.com/fallahn/tmxlite>

React Team. (2024). React 19 Documentation. [Online].
Available at: <https://react.dev/>

Appendices

Appendix A - System Architecture

- C++ Game Client - gameplay rendering, AI
- Node.js WebSocket Relay - session management, message broadcast
- ASP.NET Core API - authentication, session creation
- React PWA - spectator interface
- Communication:
 - WebSocket (real-time actions)
 - HTTP (authentication, session creation)

Appendix B - WebSocket Message Format

- action: help/hinder
- effect: gameplay effect
- user: spectator ID

Server → Client

- progress: includes hinder/help counts and unlock states
- reset: reset counters after special actions

Appendix C - Help/Hinder Actions

Help:

- heal_player
- speed_up_player
- power_boost
- shield_player
- god_mode (unlock after 5)

Hinder

- spawn_enemy
- slow_player
- steal_power
- drop_trap
- spawn_brute (unlock after 5)

Appendix D - AI Behaviour Modes

- Seek - direct movement to target
- Arrive - decelerated movement near target
- Pursue - predicted target position
- Wander - randomised movement
- Flee - movement away from target

Additional:

- Wall avoidance - removes movement into collision
- Separation - avoid bunching with other nearby NPC's.

Appendix E - Turn-Based Battle States

- PLAYER_CHOICE
- PLAYER_ACTION
- ENEMY_ACTION
- WIN
- LOSE

Player actions:

- Attack
- Magic (Fireball, Thunder, Ice, Heal)
- Item

Appendix F - Core Class Structures

- Entity - base class
- Player - movement, combat
- NPC - AI + combat logic
- Game - main loop, system control
- AIBehaviour - steering system
- FinalLevel - turn-based system